<u>Hiwonder Robot Plays Fetch</u>

By Joseph Donaldson, Brenna O'Donnell, Chiang Hui Zhi

Introduction

The goal of the project was to make the Hiwonder robot play "fetch" to some degree. This would include 2 main objectives, search for and identify a cube placed in a random but valid position, move to the cube and "retrieve" it by grabbing it and moving it to a predetermined position. This is a scalable task, so we determined an MVP and stretch goals. The MVP requires that the Hiwonder robot detects and picks up the block from within its arm radius. Stretch goals included moving to a far away block and using gesture detection to determine if the block has been thrown. These stretch goals were not implemented due to time constraints and computer vision troubles. Our team wanted to do this project to explore multiple aspects of robotics in a fun way. This project put together vision, detection, motion planning, grasp planning and how all those things combine at a scale that we could achieve in the final project time frame.

Methodology and Implementation

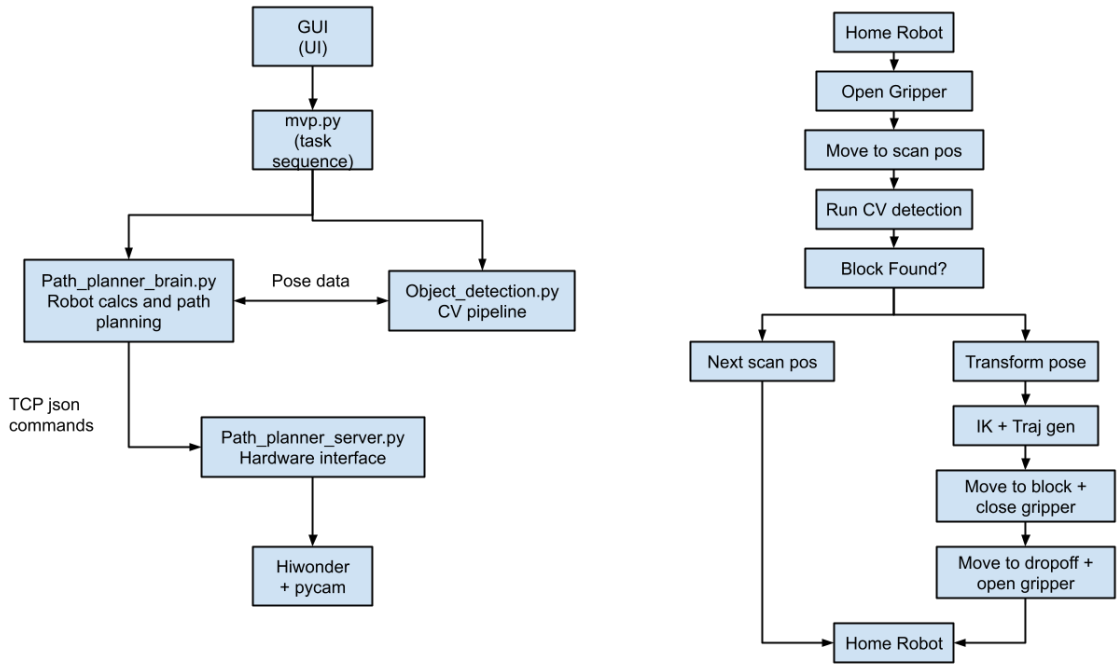
Our task follows these steps:

1. Move to home position and open the gripper
2. Scanning loop
    a. Run the CV pipeline at each of 3 scanning positions
    b. If the Aruco marker on the block has been detected, break and move to step 3
3. If the block was detected, read the block position which is returned by the CV pipeline and generate a trajectory to the block
4. Move to the block and close the gripper
5. Move to the dropoff position and open the gripper
6. Return to the home position

This task requires multiple aspects of robotics theory. The robot primarily moves using trajectory generation, which is built off of inverse kinematics, which uses the robot DH matrix. The robot uses a full computer vision pipeline to go from a warped image from the PyCam to a set of vectors representing the block rotation and position in the robot frame. We use a python server to act as the "brains" of the robot and give the robot commands on how to move. All of this is used by our main script, mvp.py

This whole process can be seen in Figure 1, and each aspect of the project will be explained in its own section below.

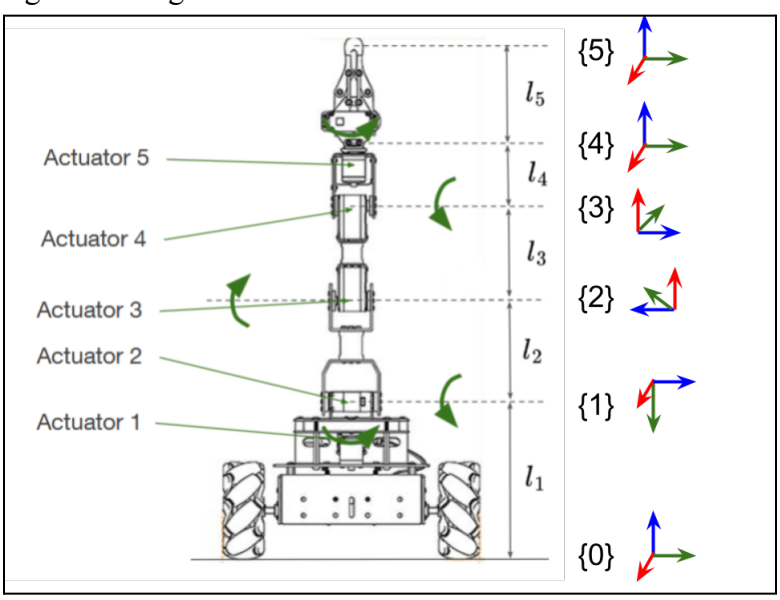
Figure 1: Block Diagram



Caption: Software and hardware architecture diagram. mvp.py coordinates task execution, while the laptop-side planning brain performs kinematics, trajectory generation, and frame transformations. The Raspberry Pi server acts as a hardware interface for the robotic arm and camera.

DH Matrices

Figure 1: Diagram of the 5DOF Hiwonder arm with coordinate frames.



Caption: Image of the front of the Hiwonder arm used in the project, labeled with the coordinate frames that correspond with each joint

Table 1: Derivation of the FPK transforms for the 5DOF Hiwonder arm

Frame	θ_i	d_i	a_i	α_i
1	θ_1	l_1	0	$-\Pi/2$
2	$\theta_2-\Pi/2$	0	l_2	Π
3	θ_3	0	l_3	Π
4	$\theta_4+\Pi/2$	0	0	$\Pi/2$
5	θ_5	l_4+l_5	0	0

The DH convention was used to create Figure 1, which was then translated to Table 1. Table 1 showcases the Denavit-Hartenberg parameters of each joint of the 5DOF Hiwonder robot used in the project. This table is the basis of all movement of the robot.

Forward Kinematics

The forward kinematics function takes in a set of joint values and uses the DH table with them to calculate the cumulative H (transformation) matrix. The H matrix is used to determine the position of the end effector.

Figure 2: calc_forward_kinematics function

```python
def calc_forward_kinematics(self, joint_values: list, radians=True):
        curr_joint_values = joint_values.copy()

        if not radians:
            curr_joint_values = [np.deg2rad(theta) for theta in curr_joint_values]

        for i, theta in enumerate(curr_joint_values):
            curr_joint_values[i] = np.clip(theta, self.joint_limits[i][0], self.joint_limits[i][1])

        th1, th2, th3, th4, th5 = curr_joint_values

        t1 = th1
        t2 = th2 - np.pi/2
        t3 = th3
        t4 = th4 + np.pi/2
        t5 = th5

        c1, s1 = cos(t1), sin(t1)
        H0_1 = np.array([
            [ c1, -s1,  0,  0         ],
            [ s1,  c1,  0,  0         ],
            [  0,   0,  1,  self.l1   ],
            [  0,   0,  0,  1         ]
        ]) @ np.array([
            [ 1,  0,  0,  0 ],
            [ 0,  0,  1,  0 ],
            [ 0, -1,  0,  0 ],
            [ 0,  0,  0,  1 ]
        ])

        #following calculations omitted for a smaller screenshot
        H1_2 = ..., H2_3 = ..., H3_4 = ..., H4_5 = ...

        Hlist = [H0_1, H1_2, H2_3, H3_4, H4_5]

        H_cumulative = [np.eye(4)]
        for i in range(self.num_dof):
            H_cumulative.append(H_cumulative[-1] @ Hlist[i])

        H_ee = H_cumulative[-1]

        ee = ut.EndEffector()
        ee.x, ee.y, ee.z = (H_ee @ np.array([0, 0, 0, 1]))[:3]

        rpy = ut.rotm_to_euler(H_ee[:3, :3])
        ee.rotx, ee.roty, ee.rotz = rpy[0], rpy[1], rpy[2]

        return ee, Hlist
```

Caption: Screenshot of the python code used to calculate the forward kinematics of the 5DOF Hiwonder robotic arm.

In this project, forward kinematics was primarily used in trajectory generation.

Inverse Kinematics

Analytic inverse kinematics was used for this project because it allows for a specified pitch of the end effector. The calc_inverse_kinematics function takes in the desired end effector position and the current joint values. It uses a set of equations to get all the possible joint combinations for a given end-effector position, then uses the robot joint limits and current joint values to determine the best of the joint possibilities.

Figure 3: calc_inverse_kinematics function

```python
def calc_inverse_kinematics(self, ee: object, init_joint_values: list, phi_d=None, soln=0) -> list:
    x, y, z = ee.x, ee.y, ee.z
    L = self.l4 + self.l5
    th5 = init_joint_values[4]

    def _wrap(a):
        return ((a + np.pi) % (2 * np.pi)) - np.pi

    def _try(r_sign, phi, sign3):
        r_ee = np.sqrt(x**2 + y**2)
        th1  = np.arctan2(y, x) + (0 if r_sign > 0 else np.pi)
        rw   = r_sign * r_ee - L * np.sin(phi)
        zw   = z             - L * np.cos(phi)
        dx, dz = rw, zw - self.l1
        d2   = dx**2 + dz**2
        c3   = (d2 - self.l2**2 - self.l3**2) / (2 * self.l2 * self.l3)
        if abs(c3) > 1.0:
            return None
        th3  = sign3 * np.arccos(np.clip(c3, -1.0, 1.0))
        beta = np.arctan2(self.l3 * np.sin(th3), self.l2 + self.l3 * np.cos(th3))
        th2  = np.arctan2(dx, dz) + beta
        th4  = phi - (th2 - th3)
        sol  = [_wrap(th1), _wrap(th2), _wrap(th3), _wrap(th4), _wrap(th5)]
        if all(self.joint_limits[i][0] <= sol[i] <= self.joint_limits[i][1] for i in range(self.num_dof)):
            return sol
        return None

    r_ee = np.sqrt(x**2 + y**2)
    phi_g = np.arctan2(r_ee, z - self.l1)
    phi_seed = init_joint_values[1] - init_joint_values[2] + init_joint_values[3]
    geo_seeds = [phi_g, -phi_g, phi_g + np.pi, phi_g - np.pi, -phi_g + np.pi, -phi_g - np.pi]
    if phi_d is not None:
        phi_list = [phi_d] + geo_seeds + [phi_seed] + list(np.linspace(-np.pi, np.pi, 36, endpoint=False))
    else:
        phi_list = geo_seeds + [phi_seed] + list(np.linspace(-np.pi, np.pi, 36, endpoint=False))

    solutions = []
    for r_sign in [1, -1]:
        for phi in phi_list:
            for sign3 in [1, -1]:
                sol = _try(r_sign, phi, sign3)
                if sol is not None and sol not in solutions:
                    solutions.append(sol)

    if not solutions:
        return []

    q0 = np.array(init_joint_values[:self.num_dof])
    if phi_d is not None:
        best = min(solutions, key=lambda s: abs((s[1] - s[2] + s[3]) - phi_d))
    else:
        best = min(solutions, key=lambda s: np.linalg.norm(np.array(s) - q0))
    return best
```

Caption: Screenshot of the python code used to calculate the analytic inverse kinematics of the 5DOF Hiwonder robotic arm.

Computer Vision

The computer vision pipeline is handled by object_detection.py, which uses ArUco markers placed on the target block. The pipeline opens the camera, captures a frame after a short delay, undistorts it using pre-calibrated intrinsic parameters K and distortion coefficients, then runs ArUco detection to find any markers in the frame. Once a marker is found, it uses cv2.solvePnP to estimate the pose of the marker relative to the camera frame, returning rotation and translation vectors. Detection is run up to 3 times and averaged to reduce noise before the result is sent back. The translation from camera frame to robot base frame is not done here, that transformation is handled by the path planner brain on the laptop side, described in the next section.

We also experimented briefly with using an LLM to interpret the scene rather than the ArUco pipeline. Both a local and an online LLM were tested, and both averaged over 5 seconds per response, which was completely unusable for a real time pick and place task. The ArUco approach stayed.

.

Python Server and Path Planner Brain

The system uses a two part architecture. path_planner_server.py runs on the Raspberry Pi on the robot and acts as a thin hardware interface, and path_planner_brain.py runs on the laptop and handles all the actual computation.

The Pi server is essentially a dummy server. It listens on a TCP socket and accepts JSON commands. When it receives joint angles it passes them directly to the HiwonderRobot interface which drives the physical servos at 20hz. It also handles gripper commands and triggers the camera for detection, returning the raw camera frame translation vectors back to the laptop. It does not do any math itself.

All the heavy lifting happens in the brain on the laptop side. When a detect result comes back from the Pi, the brain takes the raw translation vector from the camera and applies the hand-eye transform to convert it from camera frame into robot base frame coordinates in millimeters. From there, the brain runs inverse kinematics to find valid joint configurations for that target position, handles trajectory generation using cubic interpolation between joint waypoints, checks for obstacle collisions, and streams the resulting joint angle sequence down to the Pi over the socket. The brain also manages the local simulation server on port 9697, which renders a live 3D matplotlib visualization of the arm, obstacle boxes, and simulated block positions for debugging without needing the physical robot running (needed it to avoid traveling daily as a commuter).

There is also a local sim server, pp_local_server.py, which mirrors joint states being streamed from the brain into the 3D visualizer. This was useful for validating trajectories before running them on hardware.

mvp.py

mvp.py is the top level script that ties the whole pipeline together. It is loaded and called by the GUI, which holds the socket connections to both the brain and the Pi. The run function executes the fetch sequence. It starts by homing the robot, then loops through predefined scan joint configurations, moving to each one and sending a detect command through the brain to the Pi camera. If the block is found at any scan position, it breaks out with the block position now transformed into robot base frame coordinates by the brain. If nothing is found across all positions, the robot closes its gripper and returns home.

When a block is detected, the gripper opens, the brain runs IK and trajectory generation to move the end effector to the block coordinates, and the joint stream gets sent down to the Pi which executes it on the servos. There is a check after this move, if the IK solver failed to find a solution the pickup is skipped cleanly rather than crashing. If the move succeeds, the gripper closes, the brain generates a new trajectory to the dropoff position, the robot moves there, opens the gripper, and returns home.

mvp2.py is a second version with a wider set of scan positions and two extra behaviors. A celebration function that waves the arm and pulses the gripper three times after a successful pickup, and a _no_block_dance that sweeps the arm side to side a few times when no block is found, instead of going straight home.

Results

Video Link: https://youtu.be/OypeQhLTVBc?si=mlR5jEimHxC0cxbP

Reflections and Conclusions

Our MVP was a great success. The robot can consistently locate and pick up the target block when it is placed within the valid zone. The code is also set up such that given an additional week or so for the project, it could have been greatly scaled up. We could add robot base movement to not only allow a 360 degree range of areas to pick up, but we could add driving to allow for pickup anywhere. We could also add new elements to the CV pipeline which would allow for more interaction with the robot, such as identifying a hand as a dropoff location. Unfortunately, those will remain only a possibility rather than actuality due to the major challenges we faced during the project.

Camera calibration and base frame calculations was by far the biggest challenge of the project. We very quickly got the pipeline to identify blocks and move toward them, but each time the robot reached for a block, it would miss just slightly. This could be caused by a variety of issues, such as calibration, frame calculations, inverse kinematics, and hardware problems - but most likely a combination. The cause of our error turned out to be a combination of things. Lighting conditions during calibration, rotation vs translation matrices, and adding the camera offset to the DH table all contributed to our problem. Debugging took over a week and did not leave any time to expand upon the project, only time to make it as consistent and polished as possible.

Our approach was very much “lets just get some code working on the robot and we’ll fix it from there”. This approach gave us a mindset of trying to get what we already had working instead of looking for better methods of doing what we were doing, or going back to the theory and rebuilding from scratch. If we were to do this project again, we would start with the theory, writing down all the math and concepts needed then converting those to code. This approach

would have probably caught the DH table and frame conversion errors before any code was even written, and let us write our MVP code perhaps in a more efficient way.

We learned the importance of starting with the theory, and being much more careful with steps like camera calibration.